



SIMATS
ENGINEERING



SIMATS
Savitribi Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DSA0602-DATA HANDLING AND DATA VISUALIZATION

SLOT A

Dr.ALMAS BEGUM

PRESENTED BY:

SURU SAI KEERTHI [192424368]

Class Activity: 10.07.2026

Scenario 1: Student Academic Performance Dashboard

Learning Outcome

Understand how position and color aesthetics represent different data types.

Scenario

The Head of the Computer Science Department wants to analyze whether students with higher

attendance perform better in examinations. The department also wants to compare the performance across different programs.

Dataset

- Student ID
- Attendance (%)
- Internal Marks
- Department (CSE, AI, IT, ECE)

Student Task

- Identify quantitative and categorical variables.
- Map Attendance to the X-axis.
- Map Marks to the Y-axis.
- Represent the Department using different colors.
- Add an appropriate title, labels, and legend.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Student Dataset
data = {
    "Student ID": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    "Attendance": [72, 85, 91, 65, 96, 88, 78, 82, 69, 94],
    "Internal Marks": [68, 79, 91, 60, 95, 87, 73, 81, 66, 93],
    "Department": ["CSE", "AI", "IT", "ECE", "CSE", "AI", "IT", "ECE", "CSE", "AI"]
}

df = pd.DataFrame(data)

# Create Figure
plt.figure(figsize=(10,6))

# Plot department-wise
for dept in df["Department"].unique():

    dept_data = df[df["Department"] == dept]

    plt.scatter(
        dept_data["Attendance"],
        dept_data["Internal Marks"],
        s=120
    )
```

```
# Label each point
for i in range(len(dept_data)):
    plt.text(
        dept_data["Attendance"].iloc[i] + 0.3,
        dept_data["Internal Marks"].iloc[i] + 0.3,
        f'({dept_data["Student ID"].iloc[i]}, {dept})',
        fontsize=8
    )
```

```
# Title and Labels
```

```
plt.title("Student Academic Performance Dashboard", fontsize=16)
```

```
plt.xlabel("Attendance (%)", fontsize=12)
```

```
plt.ylabel("Internal Marks", fontsize=12)
```

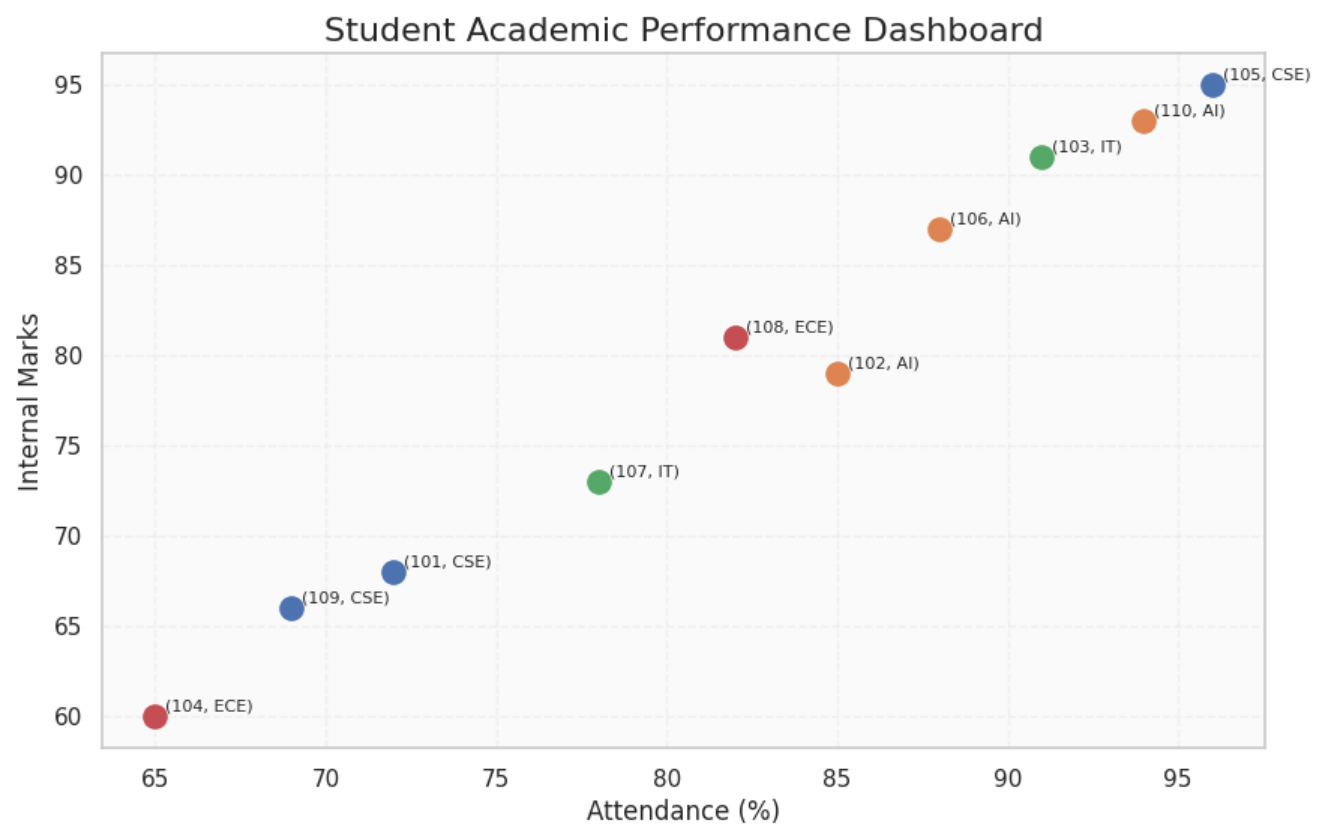
```
# Grid
```

```
plt.grid(True, linestyle="--", alpha=0.5)
```

```
# Display Graph
```

```
plt.show()
```

OUTPUT:



Scenario 2: Electric Vehicle Market Analysis

Learning Outcome

Learn how size and color aesthetics can simultaneously represent multiple variables.

Scenario

An automobile company is studying the relationship between battery capacity and vehicle price

to understand whether larger batteries significantly increase the selling price.

Dataset

- Vehicle Model
- Battery Capacity (kWh)
- Price
- Driving Range
- Manufacturer

Student Task

- Plot Battery Capacity against Price.
- Use Driving Range as bubble size.
- Use Manufacturer as color.
- Interpret whether larger batteries correspond to higher prices.

CODE:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# EV Dataset
```

```
data = {
```

```
    "Vehicle Model": ["Nexon EV", "MG ZS EV", "Kona EV", "Atto 3", "Model 3",  
                     "XUV400", "Punch EV", "Ioniq 5"],
```

```
    "Battery Capacity": [30, 50, 39, 60, 75, 40, 35, 72],
```

```
    "Price": [15, 24, 23, 34, 55, 18, 14, 46]
```

```
"Driving Range": [325, 461, 452, 521, 614, 375, 421, 631],  
"Manufacturer": ["Tata", "MG", "Hyundai", "BYD",  
                 "Tesla", "Mahindra", "Tata", "Hyundai"]  
}
```

```
df = pd.DataFrame(data)
```

```
plt.figure(figsize=(10,6))
```

```
# Bubble Chart
```

```
plt.scatter(  
    df["Battery Capacity"],  
    df["Price"],  
    s=df["Driving Range"],  
    c=range(len(df)),  
    cmap="viridis",  
    alpha=0.8,  
    edgecolors="black"  
)
```

```
# Label every bubble
```

```
for i in range(len(df)):  
    plt.text(  
        df["Battery Capacity"][i] + 0.5,  
        df["Price"][i] + 0.2,  
        f'({df["Vehicle Model"][i]}, {df["Manufacturer"][i]} )',  
        fontsize=8  
    )
```

```
plt.title("Electric Vehicle Market Analysis", fontsize=16)
```

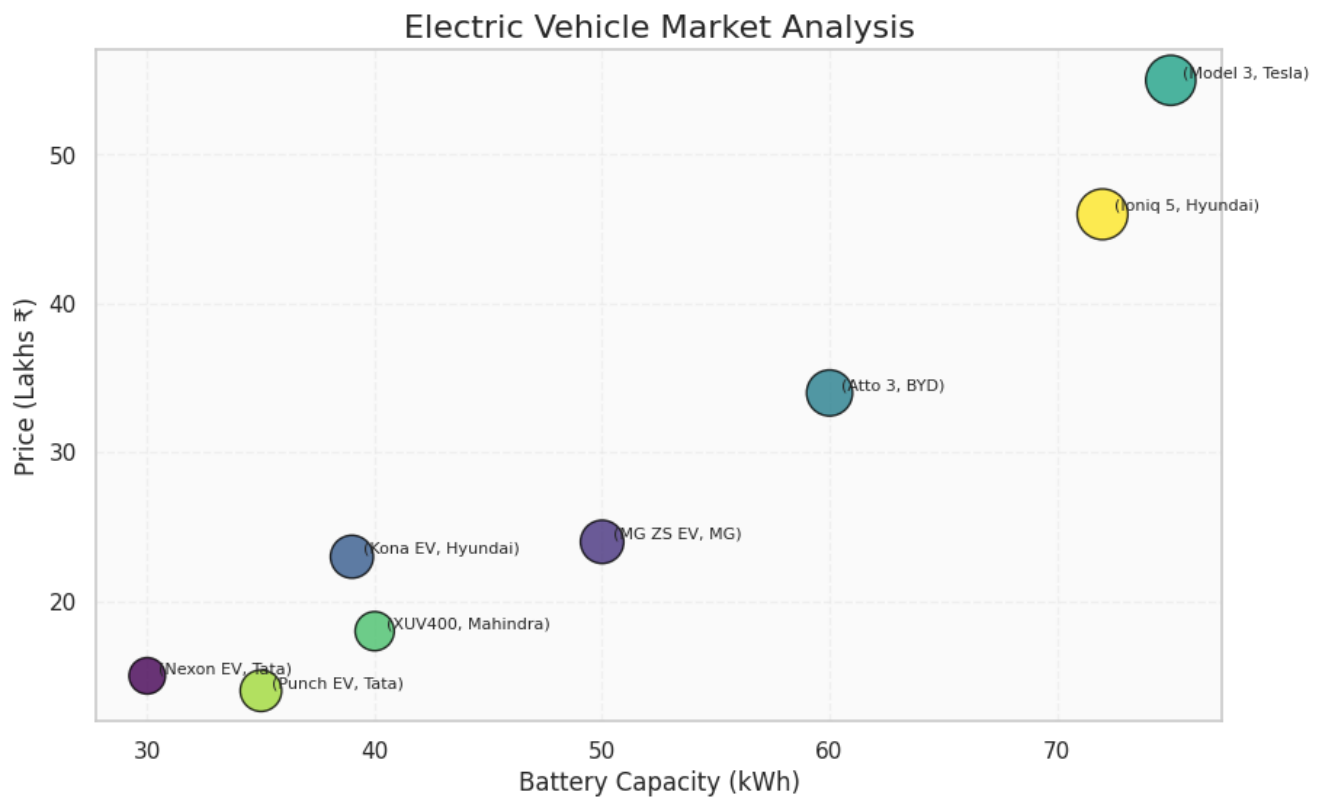
```
plt.xlabel("Battery Capacity (kWh)", fontsize=12)
```

```
plt.ylabel("Price (Lakhs ₹)", fontsize=12)
```

```
plt.grid(True, linestyle="--", alpha=0.5)
```

```
plt.show()
```

OUTPUT:



Scenario 3: Hospital Patient Admission Analysis

Learning Outcome

Visualize categorical variables using fill color and position aesthetics.

Scenario

A hospital administrator wants to compare patient admissions across departments during the

last month.

Dataset

- Department
- Number of Patients
- Average Treatment Cost

Student Task

- Create a bar chart.
- Use different fill colors for each department.
- Display treatment cost as labels on the bars.

CODE:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Hospital Dataset
```

```
data = {
```

```
    "Department": ["Cardiology", "Neurology", "Orthopedics", "Pediatrics",  
"Oncology"],
```

```
    "Number of Patients": [120, 95, 150, 110, 85],
```

```
    "Average Treatment Cost": [50000, 65000, 40000, 30000, 70000]
```

```
}
```



```
df = pd.DataFrame(data)

# Create Figure
plt.figure(figsize=(10,6))

# Bar Chart
bars = plt.bar(
    df["Department"],
    df["Number of Patients"]
)

# Display Treatment Cost on Bars
for i in range(len(df)):
    plt.text(
        i,
        df["Number of Patients"][i] + 3,
        f"₹ {df['Average Treatment Cost'][i]}",
        ha="center",
        fontsize=9
    )

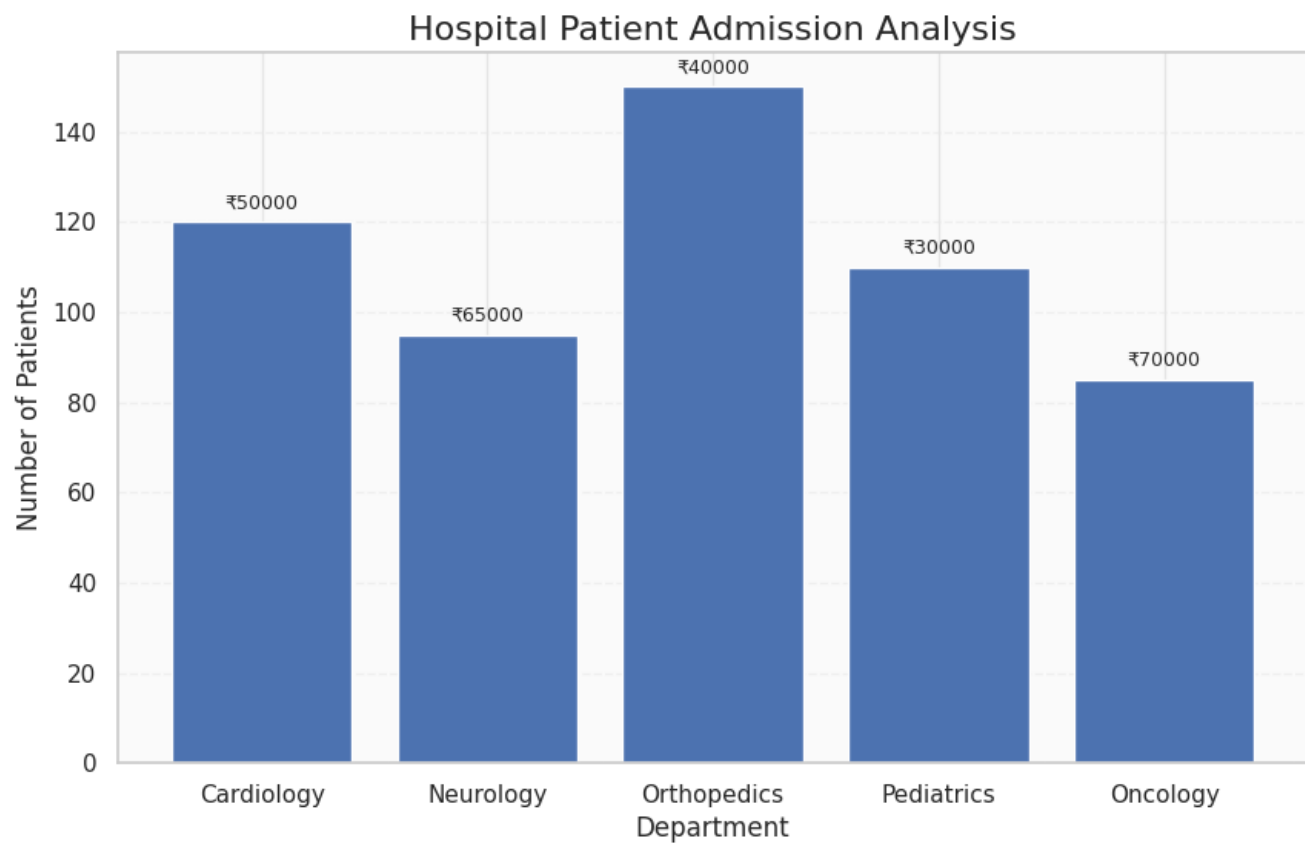
# Title and Labels
plt.title("Hospital Patient Admission Analysis", fontsize=16)
plt.xlabel("Department", fontsize=12)
plt.ylabel("Number of Patients", fontsize=12)

# Grid
```

```
plt.grid(axis="y", linestyle="--", alpha=0.5)
```

```
plt.show()
```

OUTPUT:



Scenario 4: Weather Monitoring System

Learning Outcome

Understand how line type and color distinguish multiple categories.

Scenario

The Indian Meteorological Department has collected daily temperature data for three cities

over one month and wants to compare climate trends.

Dataset

- Date
- Temperature
- City

Student Task

- Draw a line chart.
- Use different colors for each city.
- Use different line styles for each city.
- Compare temperature trends.

CODE:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Weather Dataset
```

```
data = {  
    "Day": [1,2,3,4,5,6,7],  
    "Chennai": [34,35,36,35,37,38,37],  
    "Bangalore": [28,29,30,29,31,30,29],  
    "Hyderabad": [32,33,34,35,34,36,35]  
}
```

```
df = pd.DataFrame(data)

plt.figure(figsize=(10,6))

# Chennai
plt.plot(df["Day"], df["Chennai"],
         color="blue",
         linestyle="-",
         marker="o",
         linewidth=2)

for i in range(len(df)):
    plt.text(df["Day"][i], df["Chennai"][i]+0.2,
             f'({df["Day"][i]}, {df["Chennai"][i]})',
             fontsize=8)

# Bangalore
plt.plot(df["Day"], df["Bangalore"],
         color="green",
         linestyle="--",
         marker="s",
         linewidth=2)

for i in range(len(df)):
    plt.text(df["Day"][i], df["Bangalore"][i]-0.8,
             f'({df["Day"][i]}, {df["Bangalore"][i]})',
             fontsize=8)
```

```
# Hyderabad

plt.plot(df["Day"], df["Hyderabad"],
        color="red",
        linestyle=":",
        marker="^",
        linewidth=2)

for i in range(len(df)):
    plt.text(df["Day"][i], df["Hyderabad"][i]+0.6,
            f'({df["Day"][i]}, {df["Hyderabad"][i]} )',
            fontsize=8)

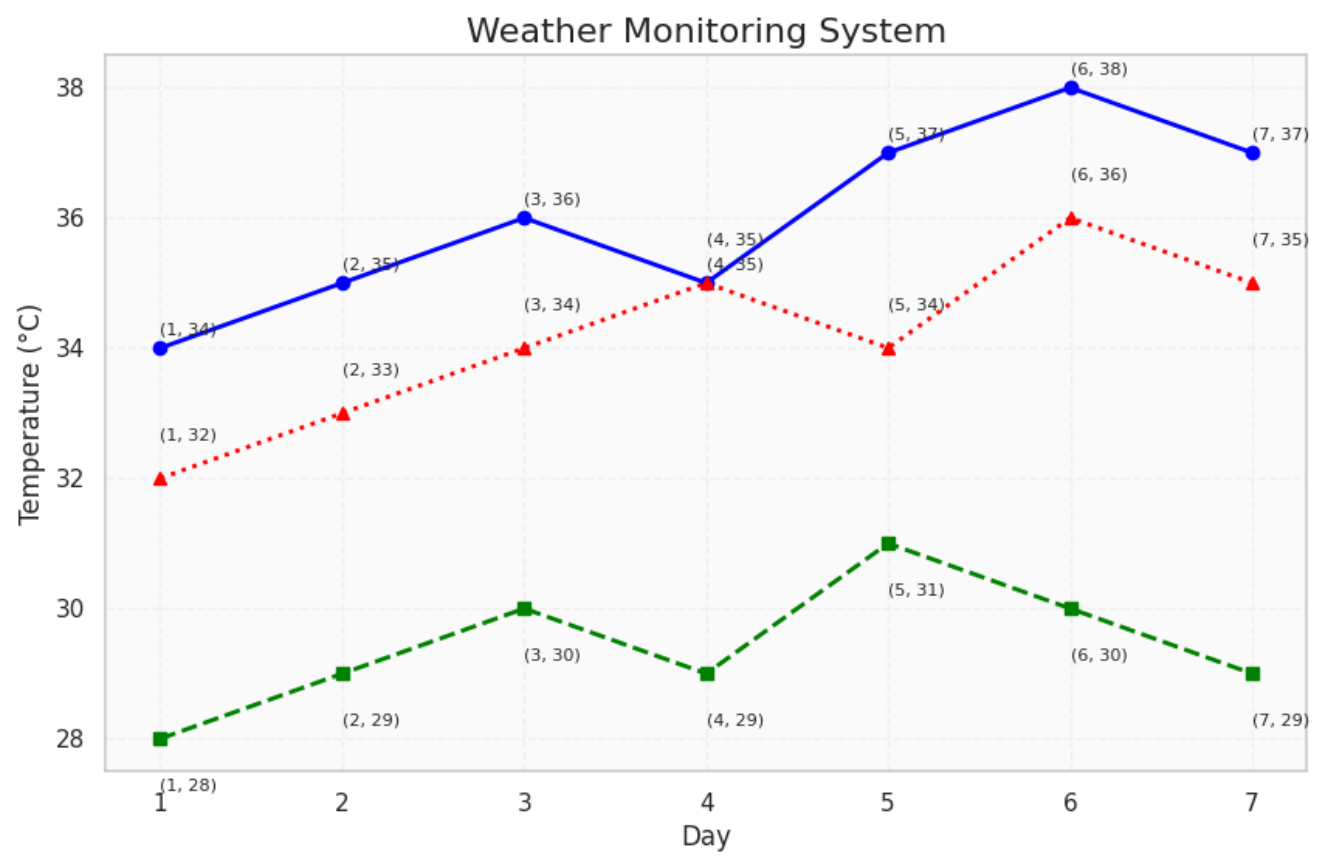
# Title and Labels

plt.title("Weather Monitoring System", fontsize=16)
plt.xlabel("Day", fontsize=12)
plt.ylabel("Temperature (°C)", fontsize=12)

plt.grid(True, linestyle="--", alpha=0.5)

plt.show()
```

OUTPUT:



Scenario 5: Supermarket Sales Analysis

Learning Outcome

Apply position and fill aesthetics for grouped comparisons.

Scenario

A supermarket chain wants to identify which product categories generate the highest monthly

sales across different branches.

Dataset

- Month
- Product Category
- Sales Amount
- Branch

Student Task

- Create grouped bar charts.
- Use branch as fill color.
- Compare product sales among branches.

CODE:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Supermarket Dataset
```

```
data = {
```

```
    "Product Category": ["Groceries", "Beverages", "Snacks", "Personal Care"],
```

```
    "Branch A": [50000, 35000, 28000, 22000],
```

```
    "Branch B": [55000, 32000, 30000, 25000],
```

```
    "Branch C": [48000, 37000, 26000, 21000]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# X-axis positions
```

```
x = np.arange(len(df["Product Category"]))
```

```
width = 0.25
```

```
plt.figure(figsize=(10,6))
```

```
# Grouped Bars
```

```
bar1 = plt.bar(x - width, df["Branch A"], width, label="Branch A")
```

```
bar2 = plt.bar(x, df["Branch B"], width, label="Branch B")
```

```
bar3 = plt.bar(x + width, df["Branch C"], width, label="Branch C")
```

```
# Labels on Bars
```

```
for bar in bar1:
```

```
    plt.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height() + 500,
        f"(A,{int(bar.get_height())})",
        ha="center",
        fontsize=8
    )
```

```
for bar in bar2:
```

```
    plt.text(
        bar.get_x() + bar.get_width()/2,
```



```
        bar.get_height() + 500,  
        f'(B,{int(bar.get_height())})",  
        ha="center",  
        fontsize=8  
    )
```

```
for bar in bar3:
```

```
    plt.text(  
        bar.get_x() + bar.get_width()/2,  
        bar.get_height() + 500,  
        f'(C,{int(bar.get_height())})",  
        ha="center",  
        fontsize=8  
    )
```

```
# Axis Labels
```

```
plt.xticks(x, df["Product Category"])  
plt.xlabel("Product Category", fontsize=12)  
plt.ylabel("Sales Amount (₹)", fontsize=12)
```

```
# Title
```

```
plt.title("Supermarket Sales Analysis", fontsize=16)
```

```
# Legend
```

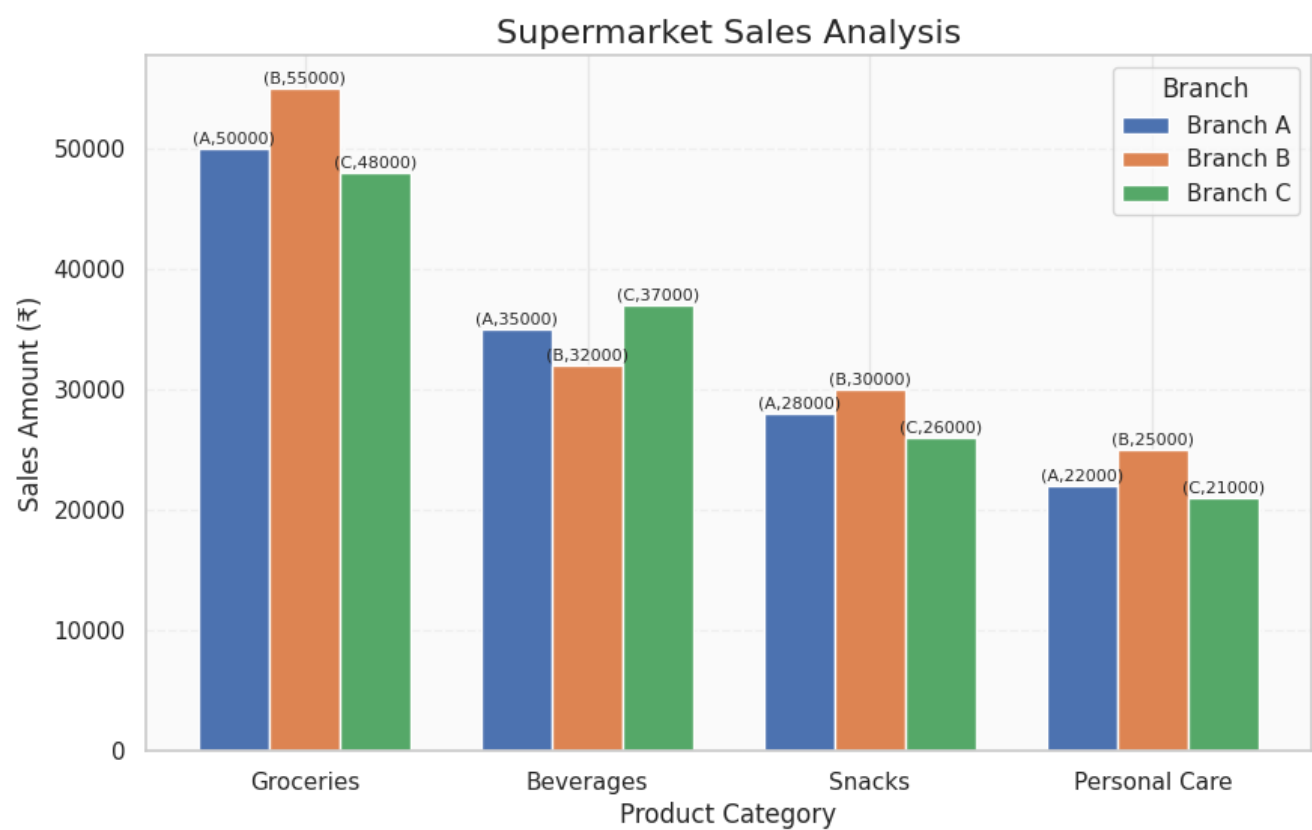
```
plt.legend(title="Branch")
```

```
# Grid
```

```
plt.grid(axis="y", linestyle="--", alpha=0.5)
```

```
plt.show()
```

OUTPUT:



Scenario 6: Employee Salary Distribution

Learning Outcome

Understand continuous scales and histogram binning.

Scenario

The HR department wants to understand salary distribution among employees to determine

whether salaries are evenly distributed across the organization.

Dataset

- Employee ID
- Salary
- Experience
- Department

Student Task

- Create a histogram of salaries.
- Experiment with different bin sizes.
- Compare the effect of changing the scale.

CODE:

```
import matplotlib.pyplot as plt
```

```
# Employee Salary Dataset (in ₹)
```

```
salary = [  
    25000,28000,30000,32000,35000,  
    37000,40000,42000,45000,47000,  
    50000,52000,55000,58000,60000,  
    62000,65000,68000,70000,72000,  
    75000,78000,80000,82000,85000,  
    88000,90000,92000,95000,98000
```

]

Histogram with 10 Bins

```
plt.figure(figsize=(8,5))
```

```
plt.hist(salary, bins=10, edgecolor="black")
```

```
plt.title("Employee Salary Distribution (10 Bins)", fontsize=15)
```

```
plt.xlabel("Salary (₹)", fontsize=12)
```

```
plt.ylabel("Number of Employees", fontsize=12)
```

```
plt.grid(axis="y", linestyle="--", alpha=0.5)
```

```
plt.show()
```

Histogram with 20 Bins

```
plt.figure(figsize=(8,5))
```

```
plt.hist(salary, bins=20, edgecolor="black")
```

```
plt.title("Employee Salary Distribution (20 Bins)", fontsize=15)
```

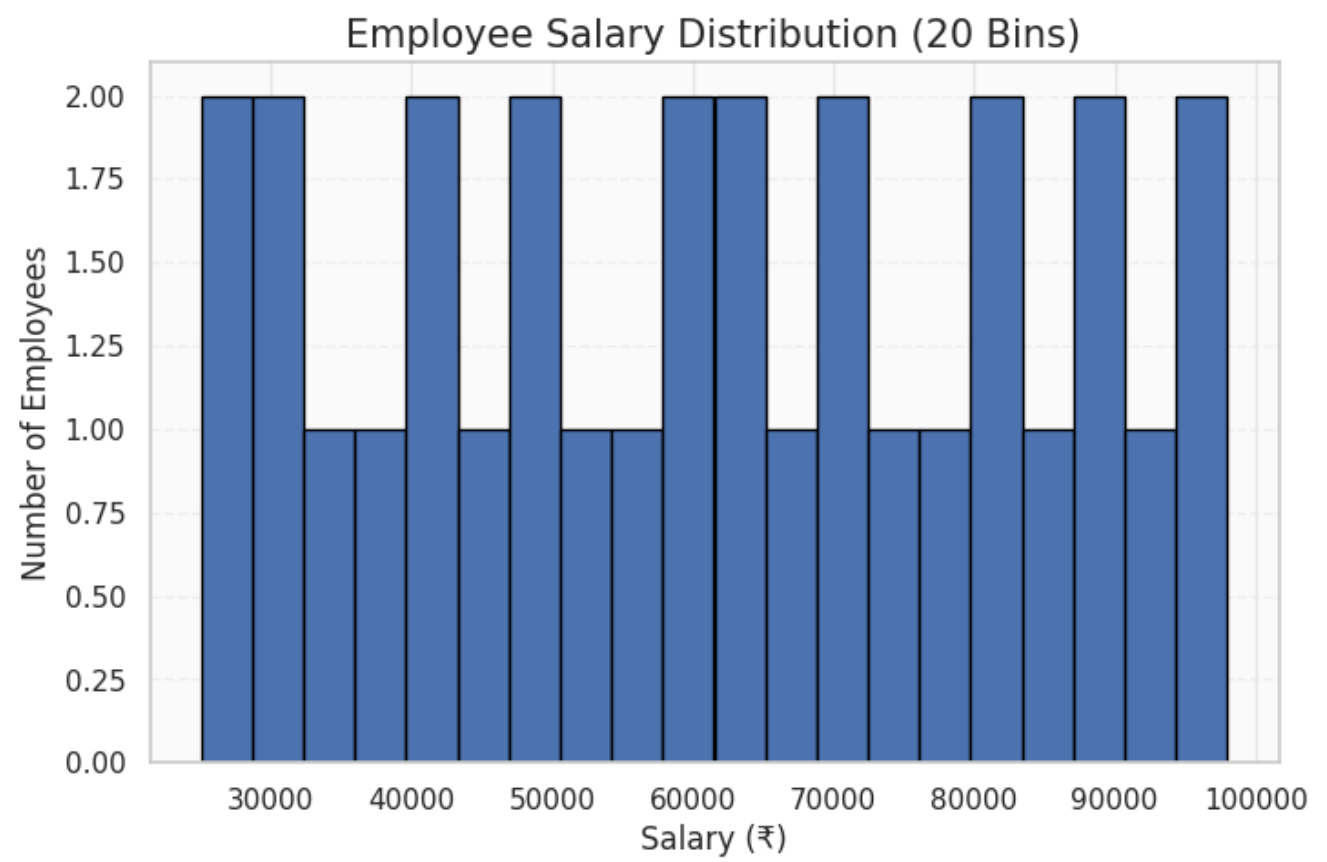
```
plt.xlabel("Salary (₹)", fontsize=12)
```

```
plt.ylabel("Number of Employees", fontsize=12)
```

```
plt.grid(axis="y", linestyle="--", alpha=0.5)
```

```
plt.show()
```

OUTPUT:



Scenario 7: Smartphone Market Comparison

Learning Outcome

Map multiple variables using shape, size, and color.

Scenario

A technology company wants to compare smartphone specifications to understand how RAM

and processor speed influence product pricing.

Dataset

- Brand
- RAM
- Processor Speed
- Price
- Storage

Student Task

- Plot RAM against Price.
- Represent Brand using shapes.
- Represent Storage using point size.
- Use color to indicate Processor Speed.

CODE:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Smartphone Dataset
```

```
data = {
```

```
    "Brand": ["Samsung", "Apple", "OnePlus", "Xiaomi", "Realme", "Vivo"],
```

```
    "RAM": [8, 6, 12, 8, 16, 12],
```

```
    "Processor Speed": [2.4, 3.2, 3.0, 2.8, 3.1, 2.0]
```

```
"Price": [35000, 75000, 50000, 28000, 45000, 38000],  
"Storage": [128, 256, 256, 128, 512, 256]  
}
```

```
df = pd.DataFrame(data)
```

```
# Marker for each Brand
```

```
markers = {  
    "Samsung": "o",  
    "Apple": "s",  
    "OnePlus": "^",  
    "Xiaomi": "D",  
    "Realme": "P",  
    "Vivo": "*"   
}
```

```
plt.figure(figsize=(10,6))
```

```
# Plot each brand
```

```
for brand in df["Brand"].unique():
```

```
    brand_data = df[df["Brand"] == brand]
```

```
    scatter = plt.scatter(  
        brand_data["RAM"],  
        brand_data["Price"],  
        s=brand_data["Storage"],          # Point Size  
        c=brand_data["Processor Speed"],  # Color
```

```
cmap="viridis",
marker=markers[brand],
edgecolors="black",
label=brand
)

# Label every point
for i in range(len(brand_data)):
    plt.text(
        brand_data["RAM"].iloc[i] + 0.1,
        brand_data["Price"].iloc[i] + 500,
        f'({brand},{brand_data["Storage"].iloc[i]}GB)',
        fontsize=8
    )
```

```
# Color Bar
plt.colorbar(scatter, label="Processor Speed (GHz)")
```

```
# Labels
plt.title("Smartphone Market Comparison", fontsize=16)
plt.xlabel("RAM (GB)", fontsize=12)
plt.ylabel("Price (₹)", fontsize=12)
```

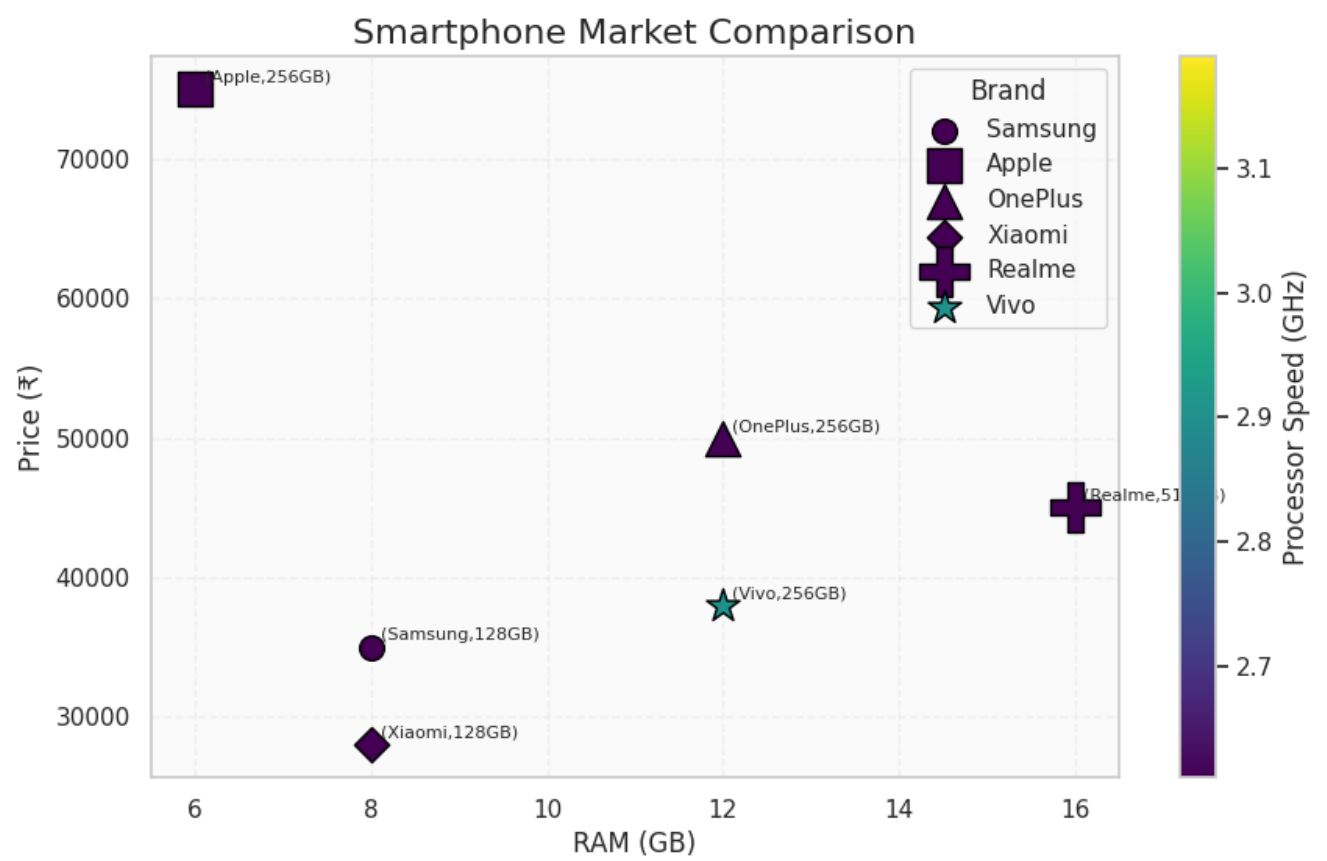
```
# Legend
plt.legend(title="Brand")
```

```
# Grid
plt.grid(True, linestyle="--", alpha=0.5)
```



```
plt.show()
```

OUTPUT:



Scenario 8: Online Movie Rating Analysis

Learning Outcome

Understand continuous color scales and gradient aesthetics.

Scenario

A movie streaming platform wants to compare average ratings received by different movie

genres over the past year.

Dataset

- Genre
- Average Rating
- Number of Reviews
- Release Year

Student Task

- Create a heatmap.
- Represent ratings using a color gradient.
- Identify the highest-rated genres.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Movie Dataset
data = {
    "Genre": ["Action", "Comedy", "Drama", "Horror", "Sci-Fi"],
    "2021": [4.2, 3.8, 4.5, 3.6, 4.7],
    "2022": [4.3, 3.9, 4.6, 3.8, 4.8],
    "2023": [4.5, 4.0, 4.7, 3.9, 4.9],
    "2024": [4.6, 4.1, 4.8, 4.0, 5.0]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# Set Genre as index
```

```
heatmap_data = df.set_index("Genre")
```

```
# Create Heatmap
```

```
plt.figure(figsize=(8,5))
```

```
sns.heatmap(  
    heatmap_data,  
    annot=True,  
    cmap="YlGnBu",  
    linewidths=1,  
    linecolor="white",  
    cbar_kws={"label": "Average Rating"}  
)
```

```
# Title and Labels
```

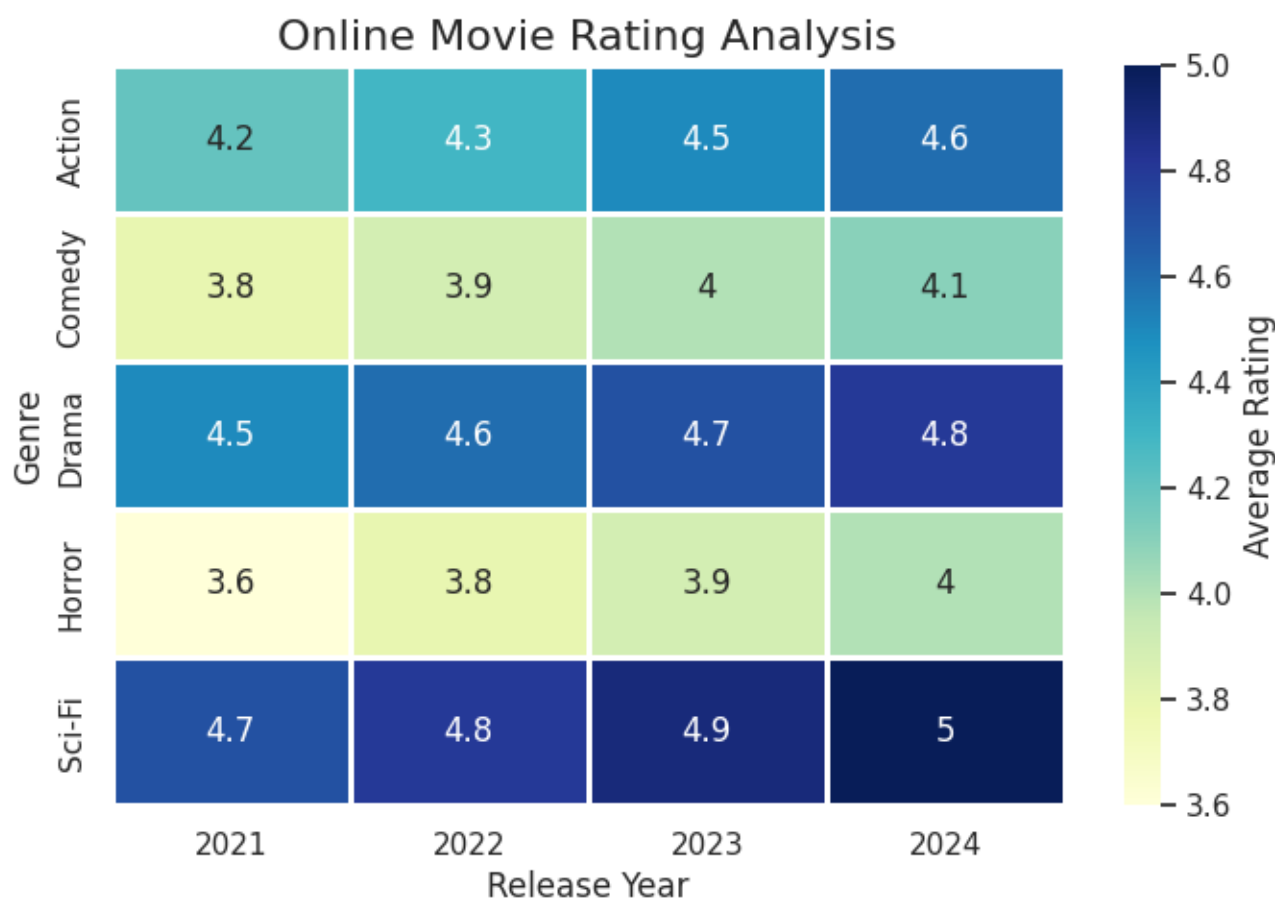
```
plt.title("Online Movie Rating Analysis", fontsize=16)
```

```
plt.xlabel("Release Year", fontsize=12)
```

```
plt.ylabel("Genre", fontsize=12)
```

```
plt.show()
```

OUTPUT:



Scenario 9: COVID-19 State-wise Analysis

Learning Outcome

Understand sequential, diverging, and categorical color scales.

Scenario

The Ministry of Health wants to visualize the distribution of COVID-19 cases across different

Indian states to identify highly affected regions.

Dataset

- State
- Active Cases
- Vaccination Percentage
- Population

Student Task

- Create a choropleth map (or tile plot if maps are unavailable).
- Represent Active Cases using a sequential color scale.
- Compare the intensity across states.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# COVID-19 Dataset
```

```
data = {
    "State": [
        "Tamil Nadu",
        "Kerala",
        "Karnataka"
```

```
"Maharashtra",  
"Delhi",  
"Telangana",  
"Andhra Pradesh",  
"Gujarat"  
],  
"Active Cases": [3200, 4500, 2800, 6200, 1800, 2100, 2600, 3000],  
"Vaccination (%)": [94, 96, 92, 91, 95, 93, 90, 89],  
"Population (Millions)": [78, 36, 69, 126, 20, 40, 54, 71]  
}
```

```
df = pd.DataFrame(data)
```

```
# Create Tile Plot (Heatmap)
```

```
heatmap_data = df.set_index("State")[["Active Cases"]]
```

```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(  
    heatmap_data,  
    annot=True,  
    cmap="Reds",  
    linewidths=1,  
    linecolor="white",  
    cbar_kws={"label": "Active Cases"}  
)
```

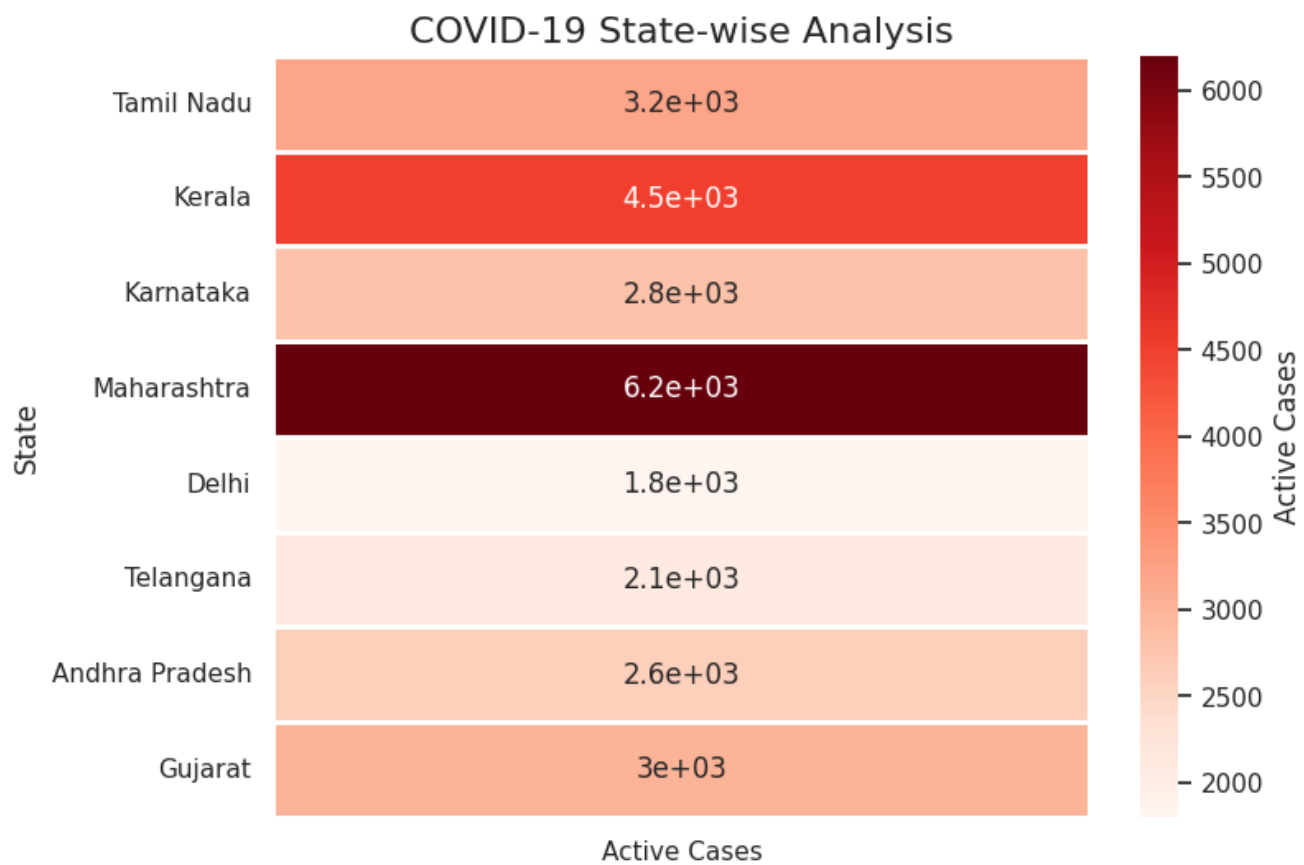
```
plt.title("COVID-19 State-wise Analysis", fontsize=16)
```

```
plt.xlabel("")
```

```
plt.ylabel("State", fontsize=12)
```

```
plt.show()
```

OUTPUT:



Scenario 10: Iris Flower Classification

Learning Outcome

Understand how multiple aesthetic mappings improve pattern recognition and class separation.

Scenario

A botanist wants to analyze flower measurements to determine whether different Iris species can be visually separated.

Dataset

(Built-in Iris Dataset)

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width
- Species

Student Task

- Plot Sepal Length against Petal Length.
- Use Species as color.
- Use Species as shape.

- Use Sepal Width as point size.
- Interpret whether species form distinct clusters.

CODE:

```
from sklearn.datasets import load_iris  
  
import pandas as pd  
  
import matplotlib.pyplot as plt
```

```
# Load Iris Dataset
```



```
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=[
        "Sepal Length",
        "Sepal Width",
        "Petal Length",
        "Petal Width"
    ]
)

df["Species"] = iris.target
df["Species"] = df["Species"].map({
    0: "Setosa",
    1: "Versicolor",
    2: "Virginica"
})

# Marker styles
markers = {
    "Setosa": "o",
    "Versicolor": "s",
    "Virginica": "^"
}

plt.figure(figsize=(10,6))
```

```
# Plot each species
for species in df["Species"].unique():

    species_data = df[df["Species"] == species]

    plt.scatter(
        species_data["Sepal Length"],
        species_data["Petal Length"],
        s=species_data["Sepal Width"] * 50, # Point Size
        c=species_data["Petal Width"],      # Color
        cmap="viridis",
        marker=markers[species],
        edgecolors="black",
        label=species
    )

# Label every point
for i in range(len(df)):
    plt.text(
        df["Sepal Length"][i] + 0.02,
        df["Petal Length"][i] + 0.02,
        df["Species"][i],
        fontsize=6
    )

# Color Bar
plt.colorbar(label="Petal Width (cm)")
```

```
# Title and Labels
```

```
plt.title("Iris Flower Classification", fontsize=16)
```

```
plt.xlabel("Sepal Length (cm)", fontsize=12)
```

```
plt.ylabel("Petal Length (cm)", fontsize=12)
```

```
# Legend
```

```
plt.legend(title="Species")
```

```
# Grid
```

```
plt.grid(True, linestyle="--", alpha=0.5)
```

```
plt.show()
```

OUTPUT:

